

使用 Java 版 Agent Development Kit (ADK) 建構 AI 代理

來源：<https://codelabs.developers.google.com/adk-java-getting-started?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何使用 Java 版 Agent Development Kit (ADK) 建構 AI 應用程式。我們將超越簡單的 LLM API 呼叫，建立自主 AI 代理，讓代理能夠推論、規劃、使用工具及共同解決複雜問題。

目錄

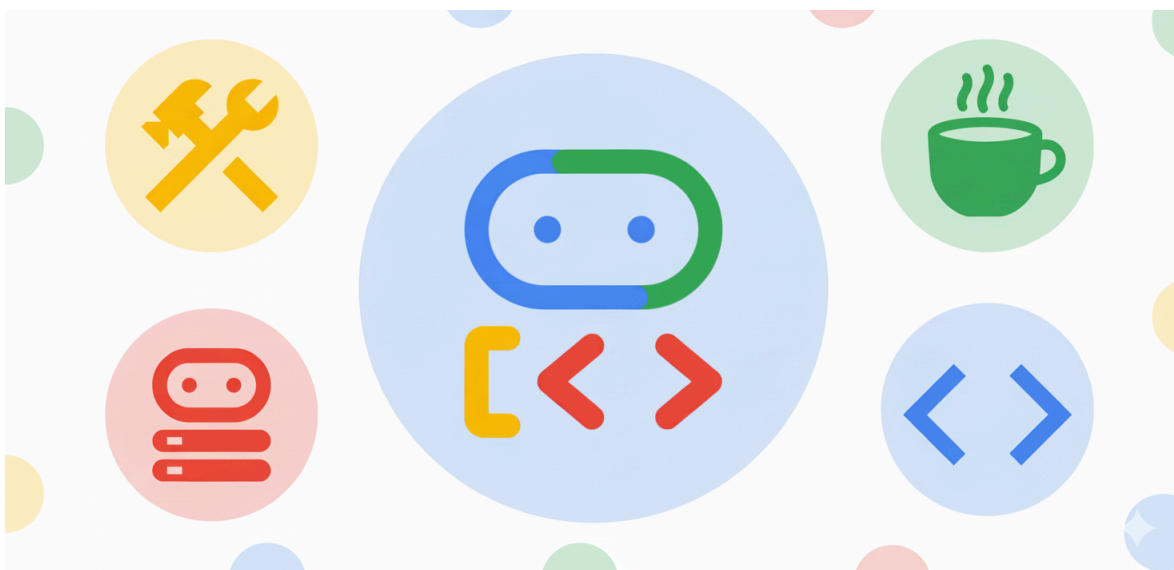
1. 歡迎，AI 代理程式開發人員！
2. 設定：您的環境
3. 開始使用：第一個代理
4. 為代理提供工具
5. Google 搜尋的強大威力，可提供最新資訊
6. 精通代理工作流程
7. 代理工作流程 - 透過子代理委派工作
8. 代理工作流程 - 生產線
9. 代理工作流程 - 同時運作
10. 代理工作流程 - 疊代式修正
11. 恭喜！

步驟 1 · 約 0 分鐘

1. 歡迎，AI 代理程式開發人員！

在本程式碼研究室中，您將瞭解如何使用 [Java 版 Agent Development Kit \(ADK\)](#)，以 **Java** 建構 **AI 代理程式**。我們將超越簡單的大型語言模型 (LLM) API 呼叫，建立自主 AI 代理，這類代理能夠推論、規劃、使用工具及共同解決複雜問題。

首先，您要兌換 Google Cloud 抵免額、設定 Google Cloud 環境，然後建構第一個簡單的代理程式，並逐步新增自訂工具、網頁搜尋和多代理程式協調等進階功能。



課程內容

- 如何建立以角色為主的基礎 AI 代理程式。
- 如何使用自訂和內建工具 (例如 Google 搜尋) 提升代理的效能。
- 如何新增以 Java 實作的工具。
- 如何將多個代理自動化調度管理為強大的循序、平行和迴圈工作流程。

軟硬體需求

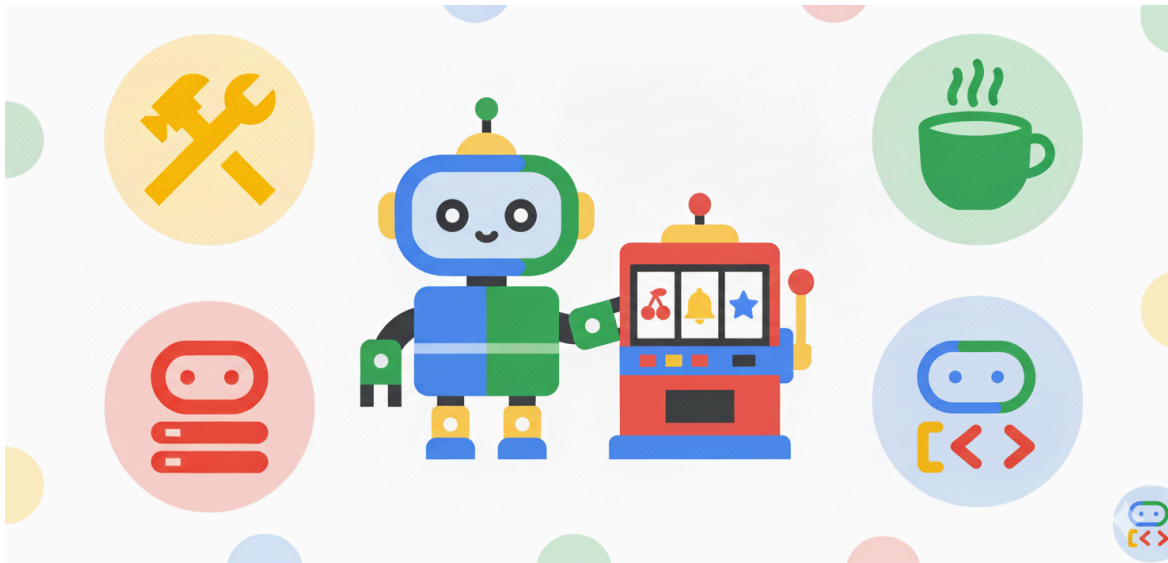
- 我們將以無痕模式使用的網路瀏覽器。
- 個人 Gmail 帳戶。
- 與個人 Gmail 帳戶相關聯的新 Google Cloud 雲端專案。
- 使用兌換的 Google Cloud 抵免額建立的帳單帳戶。
- 用於查看原始碼的 git 指令列工具。
- Java 17 以上版本和 Apache Maven。
- 文字編輯器或 IDE，例如 IntelliJ IDEA 或 VS Code。

您可以使用 Google Cloud 控制台的 Cloud Shell 內建程式碼編輯器。

進一步瞭解如何[啟動 Cloud Shell 編輯器](#)。

2. 設定：您的環境

申請研討會的 Google Cloud 抵免額



如果是講師主導的研討會，您會收到網站連結，可前往該網站申請 Google Cloud 抵免額，用於研討會。

- **使用個人 Google 帳戶：**請務必使用個人 Google 帳戶 (例如 gmail.com 地址)，公司或學校帳戶電子郵件地址無法使用。
- **使用 Google Chrome 無痕模式：**建議使用這個模式建立乾淨的工作階段，避免與其他 Google 帳戶發生衝突。
- **使用特別活動連結：**請使用活動專屬連結，格式類似 <https://trygcp.dev/event/xxx>，後面接著活動代碼 (本例為「xxx」)。
- **接受服務條款：**登入後，系統會顯示 Google Cloud Platform 服務條款，您必須接受這些條款才能繼續。
- **建立新專案：**必須從 [Google Cloud 控制台](#) 建立新的空白專案。
- **連結帳單帳戶：**將新建專案連結至帳單帳戶。
- **確認抵免額：**觀看下方影片，瞭解如何前往帳單頁面的「抵免額」部分，確認抵免額已套用至專案。

歡迎[觀看這部影片](#)，瞭解如何兌換及套用抵免額。

啟用 Vertex AI API

首先，請啟用 Vertex AI API。您可以在 Google Cloud 控制台搜尋 **Vertex AI API**，然後啟用該 API。您也可以 Cloud Shell 中執行下列指令來完成這項操作：

```
gcloud services enable aiplatform.googleapis.com
```

使用 Vertex AI 設定 ADK

如要使用 Gemini on Vertex AI 驗證 ADK AI 代理程式，以用於這個程式碼研究室，請設定下列環境變數。請務必使用專屬專案 ID。

macOS / Linux：開啟終端機並執行下列指令。如要永久啟用這項功能，請在 Shell 的啟動檔案 (例如

`~/.bash_profile`、`~/.zshrc`) 中新增這一行。

```
export GOOGLE_GENAI_USE_VERTEXAI=true  
export GOOGLE_CLOUD_PROJECT=your-project-id
```

Windows (命令提示字元)：開啟新的命令提示字元並執行：

```
set GOOGLE_GENAI_USE_VERTEXAI=true  
set GOOGLE_CLOUD_PROJECT=your-project-id
```

您必須重新啟動命令提示字元，這項變更才會生效。

Windows (PowerShell)：開啟 PowerShell 終端機並執行：

```
$env:GOOGLE_GENAI_USE_VERTEXAI = "true"  
$env:GOOGLE_CLOUD_PROJECT = "your-project-id"
```

如要在 PowerShell 中永久變更，請將其新增至設定檔指令碼。

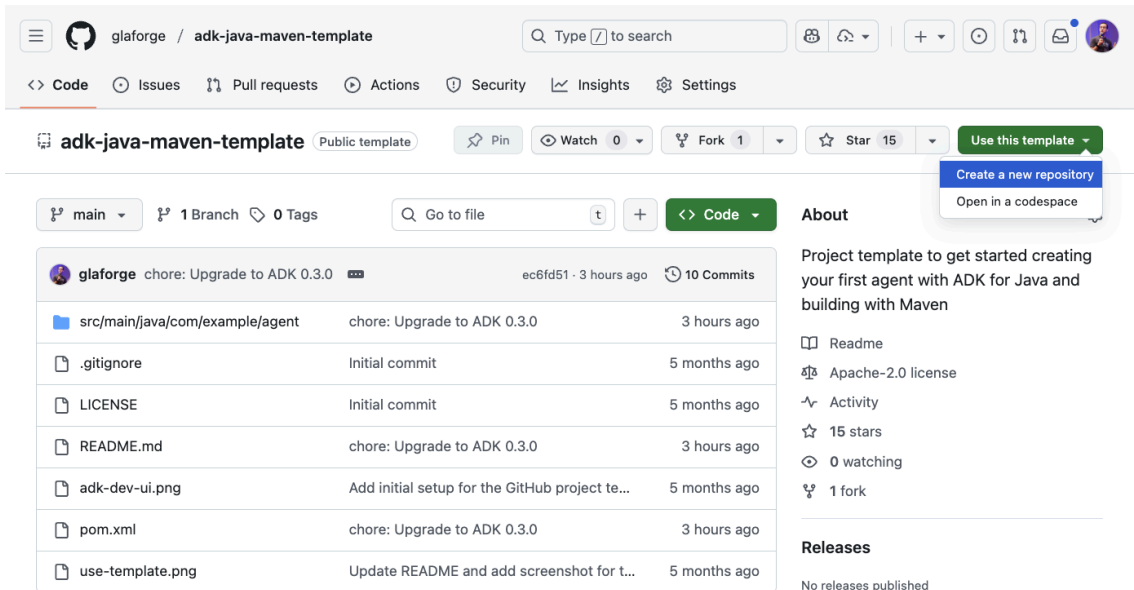
步驟 3 · 約 10 分鐘

3. 開始使用：第一個代理

如要開始新專案，最好的方法是使用 [ADK for Java GitHub 範本](#)。並提供專案結構和所有必要依附元件。

如果您有 Github 帳戶，可以依序點選 `Use this template` > `Create a new repository`，然後使用 `git clone` 指令在本地簽出程式碼。

以下螢幕截圖顯示使用範本的右上角選單。



另一種做法是直接複製該存放區：

```
git clone https://github.com/glaforge/adk-java-maven-template.git
```

如要確認您已準備好開始以 Java 程式碼建構第一個 AI 代理程式，請確保您可以使用下列指令編譯這個專案中的程式碼：

```
cd adk-java-maven-template
mvn compile
```

程式碼步驟：友善的科學老師代理程式

ADK 的基本構成元素是 `LlmAgent` 類別。這就像是具有特定個性和目標的 AI，由大型語言模型驅動。我們稍後會透過工具新增更多功能，並與其他類似代理程式攜手合作，讓這項功能更加強大。

在 `com.example.agent` 套件中建立名為 `ScienceTeacher` 的新 Java 類別。

這是建立代理程式的「Hello, World!」範例。我們將定義一個簡單的代理程式，角色是科學老師。

```
// src/main/java/com/example/agent/ScienceTeacher.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.web.AdkWebServer;

public class ScienceTeacher {
    public static void main(String[] args) {
        AdkWebServer.start(
            LlmAgent.builder()
                .name("science-teacher")
                .description("A friendly science teacher")
                .instruction("""
                    You are a science teacher for teenagers.
                    You explain science concepts in a simple, concise and direct way.
                    """)
                .model("gemini-2.5-flash")
                .build()
        );
    }
}
```

AI 代理程式是透過 `LlmAgent.builder()` 方法設定。`name()`、`description()` 和 `model()` 參數為必要參數，如要為代理提供特定個性和適當行為，請務必透過 `instruction()` 方法提供詳細指引。

我們選擇使用 Gemini 2.5 Flash 模型，但您也可以試用 Gemini 2.5 Pro 處理更複雜的工作。

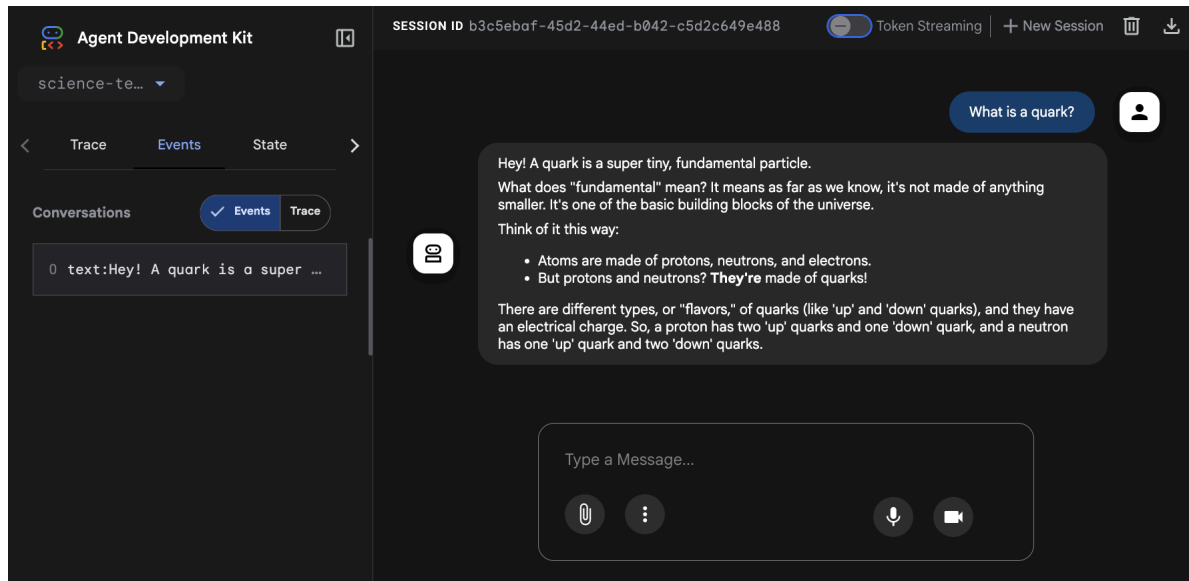
這個代理程式會包裝在 `AdkWebServer.start()` 方法中。這就是所謂的 **ADK 開發 UI** 即時通訊介面。你可以透過一般聊天介面與服務專員對話。此外，如果您想瞭解幕後運作情形 (例如流經系統的所有事件，以及傳送至 LLM 的要求和回應)，這項功能也很有幫助。

如要在本機編譯及執行這個代理程式，請執行下列指令：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.ScienceTeacher
```

然後在瀏覽器中前往 <http://localhost:8080>。如果您使用 Cloud Shell，可以前往「網頁預覽」，然後選取「透過以下通訊埠預覽：8080」。

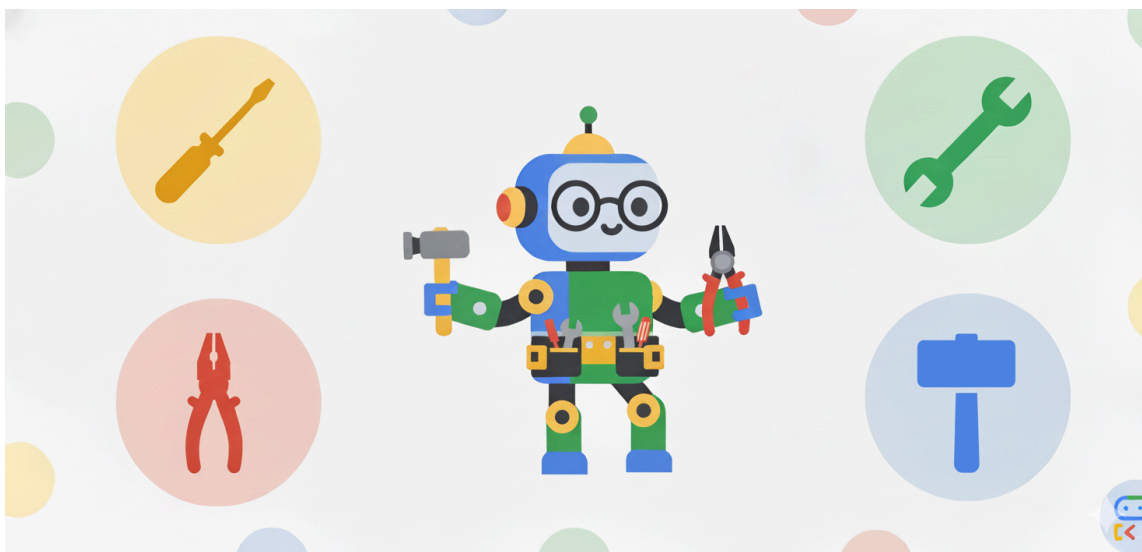
您應該會看到如下方螢幕截圖所示的 UI。請直接向虛擬服務專員提出科學相關問題。



重要概念說明：

- **instruction()**：這是代理程式的核心。這是定義代理角色、目標和限制的系統提示。如要獲得所需行為，請務必提供清楚詳盡的操作說明。
- **model()**：我們指定 "gemini-2.5-flash"，這是 Gemini 系列中強大且有效率的模型，可為代理程式的推論能力提供支援。
- **AdkWebServer.start()**：這項公用程式會啟動帶有聊天介面的本機網路伺服器，讓您立即以互動方式測試代理程式。

4. 為代理提供工具



為什麼代理商需要工具？大型語言模型功能強大，但既有知識停留在訓練當下的時間點，而且缺乏與外界互動的能力。工具就是橋樑。代理程式可藉此存取即時資訊 (例如股價或新聞)、查詢私人 API，或執行您以 Java 編寫的任何動作。

程式碼步驟：建立自訂工具 (`StockTicker`)

在這裡，我們為代理程式提供查詢股價的工具。代理程式會推論，使用者要求價格時，應呼叫我們的 Java 方法。

```
// src/main/java/com/example/agent/StockTicker.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.tools.Annotations.Schema;
import com.google.adk.tools.FunctionTool;
import com.google.adk.web.AdkWebServer;
import java.util.Map;

public class StockTicker {
    public static void main(String[] args) {
        AdkWebServer.start(
            LlmAgent.builder()
                .name("stock_agent")
                .instruction("""
                    You are a stock exchange ticker expert.
                    When asked about the stock price of a company,
                    use the `lookup_stock_ticker` tool to find the information.
                    """)
                .model("gemini-2.5-flash")
                .tools(FunctionTool.create(StockTicker.class, "lookupStockTicker"))
                .build()
        );
    }

    @Schema(
        name = "lookup_stock_ticker",
        description = "Lookup stock price for a given company or ticker"
    )
    public static Map<String, String> lookupStockTicker(
        @Schema(name = "company_name_or_stock_ticker", description = "The company name or
stock ticker")
        String ticker) {
        // ... (logic to return a stock price)
    }
}

```

如要讓代理更聰明，並賦予與世界互動的能力 (或與您自己的程式碼、API、服務等互動)，您可以透過 `tools()` 方法設定代理使用工具，尤其是自訂程式碼工具，並傳遞 `FunctionTool.create(...)`。

`FunctionTool` 需要類別和方法名稱，指向您自己的靜態方法，也可以傳遞類別的例項和該物件例項方法的名稱。

請務必透過 `@Schema` 註解，指定方法及其參數的 `name` 和 `description`，因為基礎 LLM 會使用這項資訊，判斷呼叫指定方法的時機和方式。

同樣重要的是，請提供清楚的指令，協助 LLM 瞭解如何及何時使用工具。模型或許能自行判斷，但如果您在 `instruction()` 方法中提供明確說明，函式就更有可能獲得適當呼叫。

 現在輪到您實作這個方法的邏輯！

這個方法應會傳回 `Map`。通常，這個概念是傳回含有代表結果的鍵 (例如 `stock_price`) 的對應，並將股價值與該鍵建立關聯。最後，您可以新增額外的成功 / true 鍵值組，表示作業成功。如果發生錯誤，您應傳回含有索引鍵 (例如 `error`) 的對應，並在相關聯的值中加入錯誤訊息。這有助於 LLM 瞭解呼叫是否成功，或因故失敗。

- 成功後，傳回：`{"stock_price": 123}`
- 發生錯誤時傳回：`{"error": "Impossible to retrieve stock price for XYZ"}`

說明：如果遇到困難，歡迎參考這個[範例](#)，瞭解如何實作股票代號方法。

然後執行下列指令來執行這個類別：

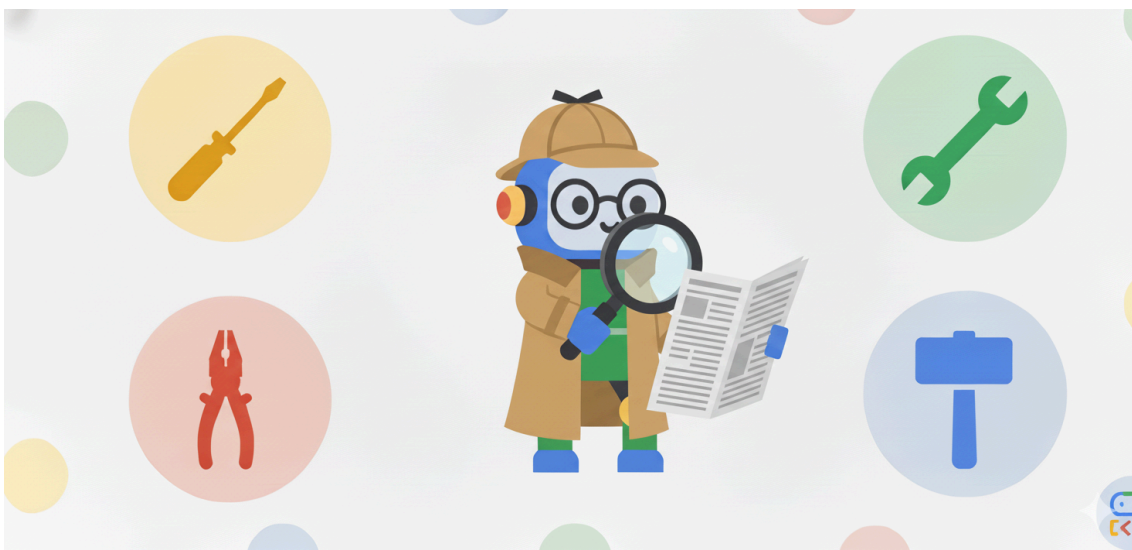
```
mvn compile exec:java -Dexec.mainClass=com.example.agent.StockTicker
```

現在可以向代理程式查詢股價，例如 `What's the stock price for GOOG?`。您應該會收到回覆，並看到系統正在使用 `lookup_stock_ticker` 工具。

基本原理和重要概念：

- `@Schema`：這個註解是魔術連結。`name` 和 `description` 會建立合約，LLM 會根據這份合約決定何時使用工具，以及提供哪些引數。
- `tools(FunctionTool.create(...))`：這會註冊已註解的 Java 方法，供代理程式使用。

5. Google 搜尋的強大威力，可提供最新資訊



Java 適用的 ADK 隨附多種強大工具，包括 `GoogleSearchTool`。有了這項工具，代理程式就能要求使用 Google 搜尋尋找相關資訊，以達成目標。

事實上，LLM 的知識會停留在特定時間點：模型會訓練到特定日期 (即「截點日期」)，使用的資料也會與收集資訊時一樣新。也就是說，LLM 可能不一定知道最近發生的事件，或知識有限且不深入，而搜尋引擎的協助或許能喚醒記憶或教導更多有關該主題的知識。

我們來看看這個簡單的新聞搜尋代理程式：

```
// src/main/java/com/example/agent/LatestNews.java
package com.example.agent;

import java.time.LocalDate;
import com.google.adk.agents.LlmAgent;
import com.google.adk.tools.GoogleSearchTool;
import com.google.adk.web.AdkWebServer;

public class LatestNews {
    public static void main(String[] args) {
        AdkWebServer.start(LlmAgent.builder()
            .name("news-search-agent")
            .description("A news search agent")
            .instruction("""
                You are a news search agent.
                Use the `google_search` tool
                when asked to search for recent events and information.
                Today is \
                """ + LocalDate.now())
            .model("gemini-2.5-flash")
            .tools(new GoogleSearchTool())
            .build());
    }
}
```

請注意，我們傳遞了搜尋工具的執行個體：`tools(new GoogleSearchTool())`。這項功能可讓代理程式掌握網路上最新的資訊。此外，提示中也指定了當天的日期，這有助於 LLM 瞭解問題是否與過去的資訊有關，以及是否需要查詢較新的資訊。

然後執行下列指令來執行這個類別：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.LatestNews
```

歡迎隨意調整提示，例如：`What's the latest news in the world?`

程式碼步驟：將搜尋代理程式做為工具

您可以建立專屬的搜尋代理，封裝搜尋功能，並將該代理做為工具公開給較高層級的代理，不必直接將 `GoogleSearchTool` 做為工具傳遞給代理。

這項進階概念可讓您將複雜的行為 (例如搜尋和摘要結果) 委派給專用子代理。這種做法通常適用於較複雜的工作流程。

```
// src/main/java/com/example/agent/SearchAgentAsTool.java
package com.example.agent;

import java.time.LocalDate;

import com.google.adk.agents.LlmAgent;
import com.google.adk.tools.AgentTool;
import com.google.adk.tools.GoogleSearchTool;
import com.google.adk.web.AdkWebServer;

public class SearchAgentAsTool {
    public static void main(String[] args) {
        // 1. Define the specialized Search Agent
        LlmAgent searchAgent = LlmAgent.builder()
            .name("news-search-agent-tool")
            .description("Searches for recent events and provides a concise summary.")
            .instruction("""
                You are a concise information retrieval specialist.
                Use the `google_search` tool to find information.
                Always provide the answer as a short,
                direct summary, without commentary.
                Today is \
                """ + LocalDate.now())
            .model("gemini-2.5-flash")
            .tools(new GoogleSearchTool()) // This agent uses the Google Search Tool
            .build();

        // 2. Wrap the Search Agent as a Tool
        AgentTool searchTool = AgentTool.create(searchAgent);

        // 3. Define the Main Agent that uses the Search Agent Tool
        AdkWebServer.start(LlmAgent.builder()
            .name("main-researcher")
            .description("Main agent for answering complex, up-to-date questions.")
            .instruction("""
                You are a sophisticated research assistant.
                When the user asks a question that requires up-to-date or external
information,
                you MUST use the `news-search-agent-tool` to get the facts before answering.
                After the tool returns the result, synthesize the final answer for the user.
                """)
            .model("gemini-2.5-flash")
            .tools(searchTool) // This agent uses the Search Agent as a tool
            .build()
        );
    }
}

```

`AgentTool.create(searchAgent)` 行是這裡的關鍵概念。系統會將整個 `searchAgent` (包含本身的內部邏輯、提示和工具) 註冊為 `mainAgent` 的單一可呼叫工具。這有助於提升模組化和可重複使用性。

使用下列指令執行這個類別：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.SearchAgentAsTool
```

如果問題很普通，代理程式會從自己的知識庫回覆，但如果詢問最近發生的事件，代理程式會使用 Google 搜尋工具，將搜尋工作委派給專門的搜尋代理程式。

6. 精通代理工作流程

如果問題很複雜，單一服務專員可能無法解決。如果目標包含過多子工作、提示詞過長且詳細，或是可存取大量函式，LLM 就會難以處理，效能和準確率也會降低。

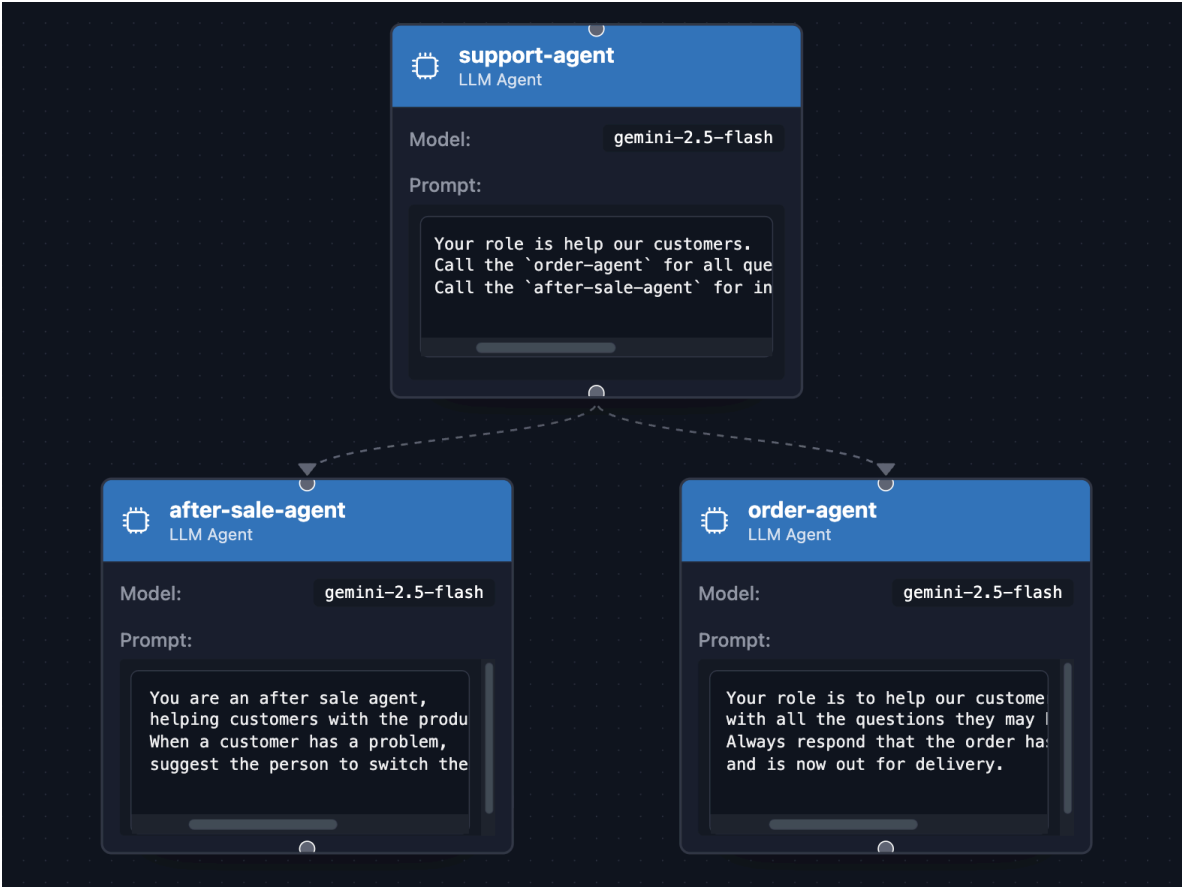
關鍵在於自動調度管理多個專用代理，**分而治之**。幸好，ADK 內建多種專用代理：

- 可將工作委派給 `subAgent()` 的一般代理
- `SequentialAgent`，依序完成工作，
- `ParallelAgent`，可平行執行代理程式，
- `LoopAgent`，通常是為了視需要多次進行精修程序。

若要瞭解主要用途，以及各工作流程的優缺點，請參閱下表。但請注意，真正強大的力量來自於結合多種工具！

工作 流程	ADK 類別	用途	優點	缺點
子代理	<code>LlmAgent</code>	使用者主導的彈性工作，不一定知道下一步該怎么做。	彈性高、對話式，非常適合面向使用者的機器人。	較難預測，依賴 LLM 推理來控制流程。
循序	<code>SequentialAgent</code>	順序至關重要的固定多步驟程序。	可預測、可靠、容易偵錯，並保證順序。	不夠彈性，如果工作可以平行處理，速度可能會較慢。
平行摺	<code>ParallelAgent</code>	從多個來源收集資料，或執行獨立工作。	效率極高，可大幅縮短 I/O 繫結工作的延遲時間。	所有工作都會執行，不會短路。不適合有依附元件的工作。
循環播放	<code>LoopAgent</code>	反覆修正、自我修正，或重複執行程序直到符合條件為止。	可解決複雜問題，並協助代理提升工作效率。	如果設計不當，可能會導致無限迴圈 (請務必使用 <code>maxIterations</code> ！)。

7. 代理工作流程 - 透過子代理委派工作



主管代理可以將特定工作委派給子代理。舉例來說，電子商務網站的服務專員可能會將訂單相關問題委派給一位專員 (「我的訂單狀態為何？」)，並將售後服務問題委派給另一位專員 (「我不知道如何開啟！」)。這就是我們要討論的使用情況。

```
// src/main/java/com/example/agent/SupportAgent.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.web.AdkWebServer;

public class SupportAgent {
    public static void main(String[] args) {
        LlmAgent orderAgent = LlmAgent.builder()
            .name("order-agent")
            .description("Order agent")
            .instruction("""
                Your role is to help our customers
                with all the questions they may have about their orders.
                Always respond that the order has been received, prepared,
                and is now out for delivery.
            """)
            .model("gemini-2.5-flash")
            .build();

        LlmAgent afterSaleAgent = LlmAgent.builder()
            .name("after-sale-agent")
            .description("After sale agent")
            .instruction("""
                You are an after sale agent,
                helping customers with the product they received.
                When a customer has a problem,
                suggest the person to switch the product off and on again.
            """)
            .model("gemini-2.5-flash")
            .build();

        AdkWebServer.start(LlmAgent.builder()
            .name("support-agent")
            .description("Customer support agent")
            .instruction("""
                Your role is to help our customers.
                Call the `order-agent` for all questions related to order status.
                Call the `after-sale-agent` for inquiries about the received product.
            """)
            .model("gemini-2.5-flash")
            .subAgents(afterSaleAgent, orderAgent)
            .build()
        );
    }
}
```

這裡的重點是呼叫 `subAgents()` 方法，並傳入兩個子代理程式，這兩個子代理程式會分別處理各自的特定角色。

使用下列指令執行上述範例：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.SupportAgent
```

你可以提出 `I have a question about my order` 或 `I have a question about my received order` 等問題。視提出的問題而定，您應該會看到主要代理將工作委派給合適的代理。

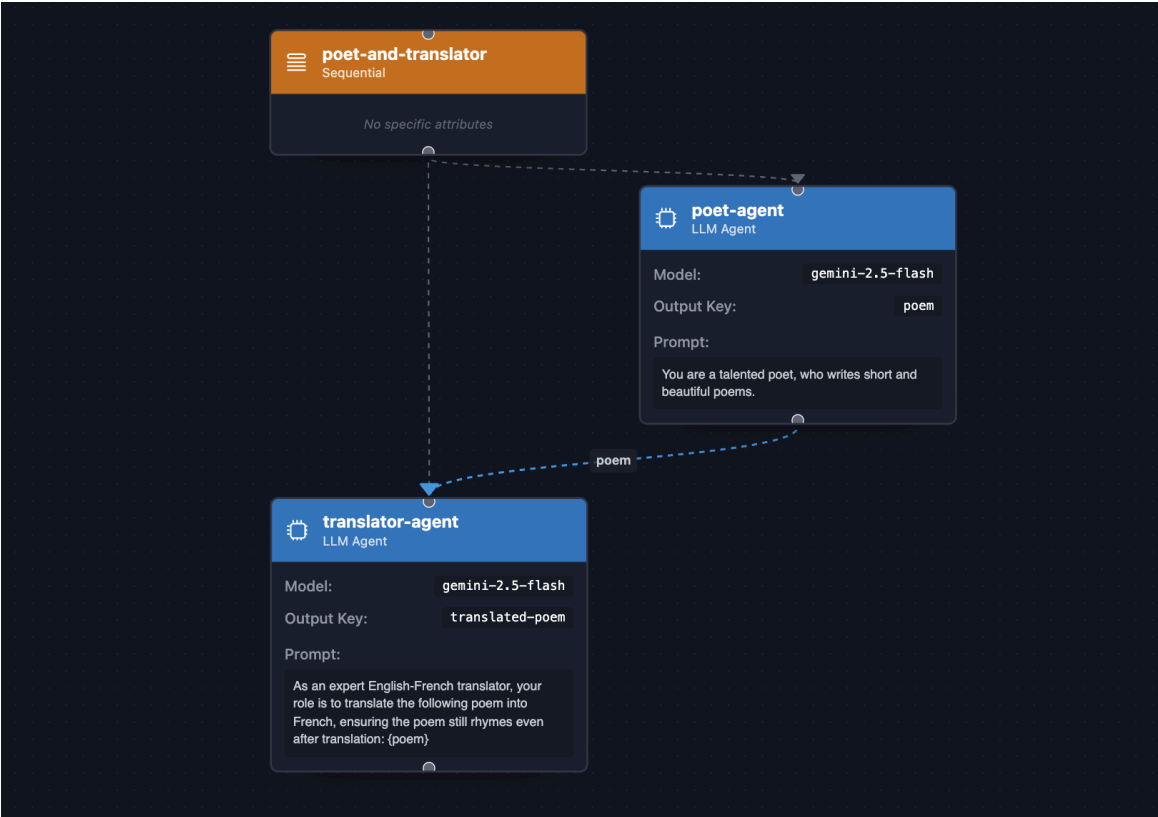
將工作委派給子代理的概念，與有效的人力管理方式相仿，優秀的經理 (主管代理) 會指派專門的員工 (子代理) 處理他們更擅長的特定工作。主管不必瞭解每個程序的細節，而是智慧地將顧客要求 (例如訂單查詢或技術問題) 轉送給最符合資格的「團隊成員」，確保回覆品質更高、效率更佳，而不是由一般專員單獨處理。此外，這些子代理程式可以完全專注於個別指派的工作，不必瞭解複雜的整體程序。

重要概念：

- **子代理**：這裡的重點是呼叫 `subAgents()` 方法，並傳入兩個子代理，這兩個子代理會分別處理特定角色。

步驟 8 · 約 10 分鐘

8. 代理工作流程 - 生產線



如果運算順序很重要，請使用 **SequentialAgent**。這就像組裝線，依固定順序執行子代理，每個步驟都可能取決於前一個步驟。

假設一位英文詩人與英法翻譯人員合作，先以英文創作詩作，再翻譯成法文：

```
// src/main/java/com/example/agent/PoetAndTranslator.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.agents.SequentialAgent;
import com.google.adk.web.AdkWebServer;

public class PoetAndTranslator {
    public static void main(String[] args) {
        LlmAgent poet = LlmAgent.builder()
            .name("poet-agent")
            .description("Poet writing poems")
            .model("gemini-2.5-flash")
            .instruction("""
                You are a talented poet,
                who writes short and beautiful poems.
                """)
            .outputKey("poem")
            .build();

        LlmAgent translator = LlmAgent.builder()
            .name("translator-agent")
            .description("English to French translator")
            .model("gemini-2.5-flash")
            .instruction("""
                As an expert English-French translator,
                your role is to translate the following poem into French,
                ensuring the poem still rhymes even after translation:

                {poem}
                """)
            .outputKey("translated-poem")
            .build();

        AdkWebServer.start(SequentialAgent.builder()
            .name("poet-and-translator")
            .subAgents(poet, translator)
            .build());
    }
}
```

執行範例：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.PoetAndTranslator
```

現在可以向代理提出類似 `Write me a poem about a lonely software developer` 的問題。系統會先以英文生成詩，然後翻譯成法文。

這種系統性做法可將複雜工作分解為較小的有序子工作，確保程序更具決定性且可靠，與依賴單一用途廣泛的代理程式相比，大幅提高成功機率。

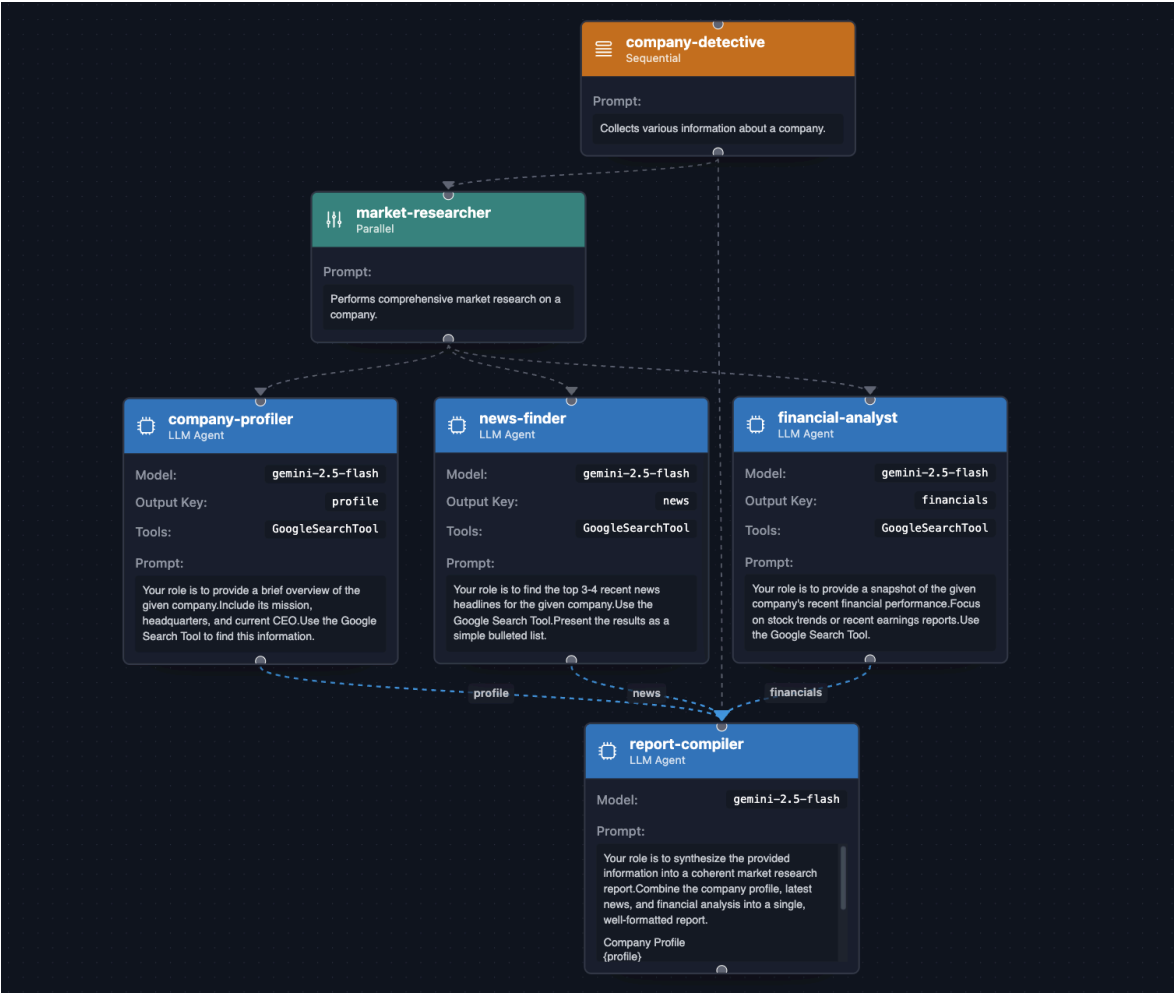
盡可能將工作分解為一系列子工作 (如果可行且合理)，對於獲得更具確定性的成功結果至關重要，因為這樣可以有條不紊地逐步完成工作，並管理各步驟之間的依附關係。

重要概念：

- **共用狀態：** `outputKey("key")` 會將代理的結果儲存至共用狀態，並使用 `{key}` 預留位置將結果插入下一個代理的指令。
-

步驟 9 · 約 10 分鐘

9. 代理工作流程 - 同時運作



如果工作彼此獨立， **ParallelAgent** 就能同時執行這些工作，大幅提升效率。在下列範例中，我們甚至會合併 **SequentialAgent** 和 **ParallelAgent**：平行工作會先執行，然後最終代理程式會彙整平行工作的結果。

我們來建立公司偵探，負責搜尋以下資訊：

- 公司簡介 (執行長、總部、座右銘等)
- 該公司的最新消息。
- 公司的財務詳細資料。

```
// src/main/java/com/example/agent/CompanyDetective.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.agents.ParallelAgent;
import com.google.adk.agents.SequentialAgent;
import com.google.adk.tools.GoogleSearchTool;
import com.google.adk.web.AdkWebServer;

public class CompanyDetective {
    public static void main(String[] args) {
        var companyProfiler = LlmAgent.builder()
            .name("company-profiler")
            .description("Provides a general overview of a company.")
            .instruction("""
                Your role is to provide a brief overview of the given company.
                Include its mission, headquarters, and current CEO.
                Use the Google Search Tool to find this information.
                """)
            .model("gemini-2.5-flash")
            .tools(new GoogleSearchTool())
            .outputKey("profile")
            .build();

        var newsFinder = LlmAgent.builder()
            .name("news-finder")
            .description("Finds the latest news about a company.")
            .instruction("""
                Your role is to find the top 3-4 recent news headlines for the given company.
                Use the Google Search Tool.
                Present the results as a simple bulleted list.
                """)
            .model("gemini-2.5-flash")
            .tools(new GoogleSearchTool())
            .outputKey("news")
            .build();

        var financialAnalyst = LlmAgent.builder()
            .name("financial-analyst")
            .description("Analyzes the financial performance of a company.")
            .instruction("""
                Your role is to provide a snapshot of the given company's recent financial
                performance.
                Focus on stock trends or recent earnings reports.
                Use the Google Search Tool.
                """)
            .model("gemini-2.5-flash")
            .tools(new GoogleSearchTool())
            .outputKey("financials")
            .build();

        var marketResearcher = ParallelAgent.builder()

```



```

        .name("market-researcher")
        .description("Performs comprehensive market research on a company.")
        .subAgents(
            companyProfiler,
            newsFinder,
            financialAnalyst
        )
        .build();

var reportCompiler = LlmAgent.builder()
    .name("report-compiler")
    .description("Compiles a final market research report.")
    .instruction("""
        Your role is to synthesize the provided information into a coherent market
research report.

        Combine the company profile, latest news, and financial analysis into a
single, well-formatted report.

        ## Company Profile
        {profile}

        ## Latest News
        {news}

        ## Financial Snapshot
        {financials}
        """)
    .model("gemini-2.5-flash")
    .build();

AdkWebServer.start(SequentialAgent.builder()
    .name("company-detective")
    .description("Collects various information about a company.")
    .subAgents(
        marketResearcher,
        reportCompiler
    ).build());
}
}

```

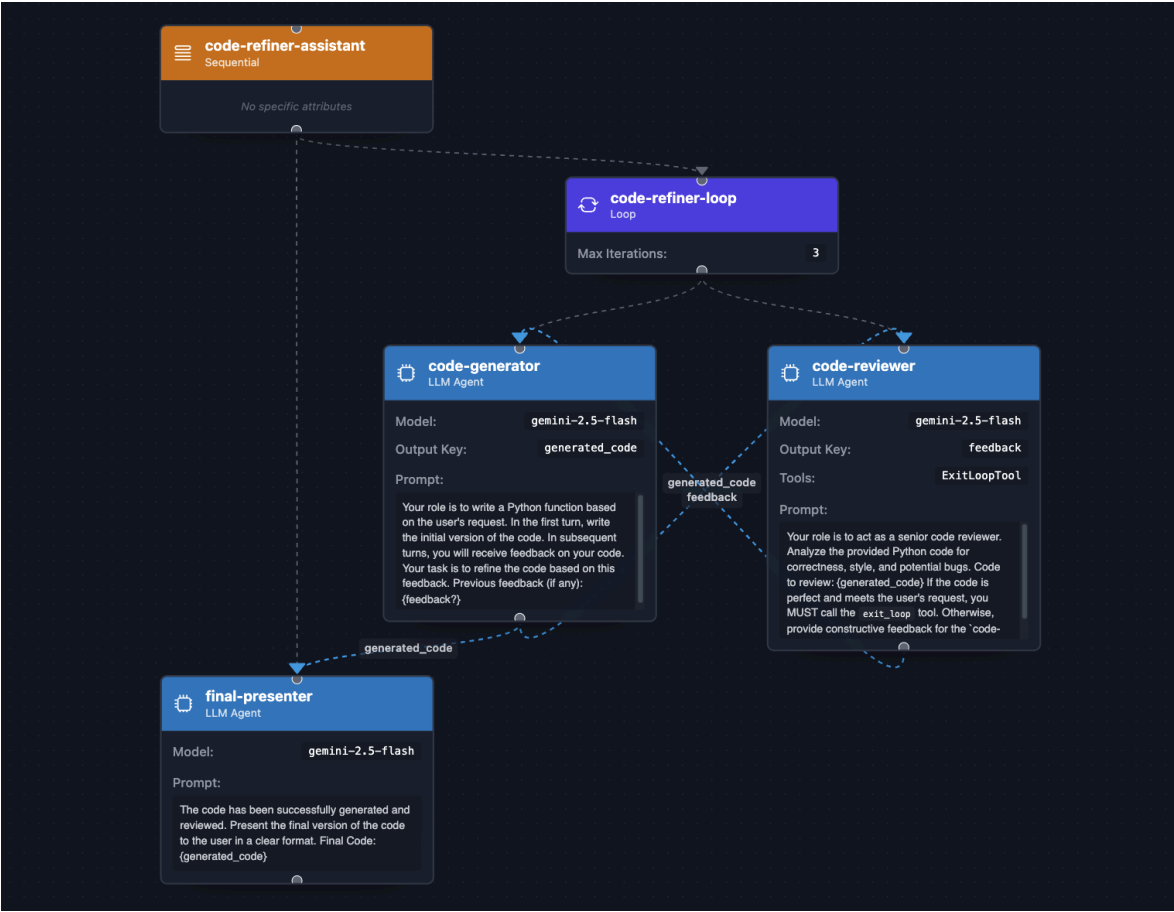
如常執行下列指令，即可執行代理程式：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.CompanyDetective
```

詢問 Google 或 Apple 等不同公司，看看會得到什麼結果。

這個代理程式展示了強大的工作流程組合，可有效運用平行和循序代理程式，並透過資訊研究和綜合的平行處理，提高效率。

10. 代理工作流程 - 疊代式修正



如果工作需要「生成 → 審查 → 修正」週期，請使用 **LoopAgent**。這項功能會自動反覆改良，直到達成目標為止。與 **SequentialAgent** 類似，**LoopAgent** 會依序呼叫子代理程式，但會從頭開始迴圈。代理內部使用的 LLM 會決定是否要求呼叫特殊工具 (即 `exit_loop` 內建工具)，以停止迴圈執行。

以下程式碼修正工具範例使用 **LoopAgent** 自動修正程式碼：生成、檢查、修正。這與人類的發展過程類似。程式碼生成器會先產生要求的程式碼，然後儲存在代理程式狀態的 `generated_code` 鍵下。程式碼審查員接著會審查生成的程式碼，並提供意見回饋 (位於 `feedback` 鍵下方)，或呼叫結束迴圈工具，提早結束疊代。

我們來看看程式碼：

```
// src/main/java/com/example/agent/CodeRefiner.java
package com.example.agent;

import com.google.adk.agents.LlmAgent;
import com.google.adk.agents.LoopAgent;
import com.google.adk.agents.SequentialAgent;
import com.google.adk.tools.ExitLoopTool;
import com.google.adk.web.AdkWebServer;

public class CodeRefiner {
    public static void main(String[] args) {
        var codeGenerator = LlmAgent.builder()
            .name("code-generator")
            .description("Writes and refines code based on a request and feedback.")
            .instruction("""
                Your role is to write a Python function based on the user's request.
                In the first turn, write the initial version of the code.
                In subsequent turns, you will receive feedback on your code.
                Your task is to refine the code based on this feedback.

                Previous feedback (if any):
                {feedback?}
                """)
            .model("gemini-2.5-flash")
            .outputKey("generated_code")
            .build();

        var codeReviewer = LlmAgent.builder()
            .name("code-reviewer")
            .description("Reviews code and decides if it's complete or needs more work.")
            .instruction("""
                Your role is to act as a senior code reviewer.
                Analyze the provided Python code for correctness, style, and potential bugs.

                Code to review:
                {generated_code}

                If the code is perfect and meets the user's request,
                you MUST call the `exit_loop` tool.

                Otherwise, provide constructive feedback for the `code-generator` to improve
                the code.

                """)
            .model("gemini-2.5-flash")
            .outputKey("feedback")
            .tools(ExitLoopTool.INSTANCE)
            .build();

        var codeRefinerLoop = LoopAgent.builder()
            .name("code-refiner-loop")
            .description("Iteratively generates and reviews code until it is correct.")
            .subAgents(
```

```

        codeGenerator,
        codeReviewer
    )
    .maxIterations(3) // Safety net to prevent infinite loops
    .build();

var finalPresenter = LlmAgent.builder()
    .name("final-presenter")
    .description("Presents the final, accepted code to the user.")
    .instruction("""
        The code has been successfully generated and reviewed.
        Present the final version of the code to the user in a clear format.

        Final Code:
        {generated_code}
        """)
    .model("gemini-2.5-flash")
    .build();

AdkWebServer.start(SequentialAgent.builder()
    .name("code-refiner-assistant")
    .description("Manages the full code generation and refinement process.")
    .subAgents(
        codeRefinerLoop,
        finalPresenter)
    .build());
}
}

```

執行下列指令來執行這個代理程式：

```
mvn compile exec:java -Dexec.mainClass=com.example.agent.CodeRefiner
```

您可以要求實作 `Tower of Hanoi` 或 `Quicksort Algorithm` 等複雜項目。

如要解決需要反覆改良和自我修正的問題，就必須使用 `LoopAgent` 實作意見回饋/修正迴圈，這與人類的認知過程十分相似。這項設計模式特別適合用於初始輸出內容很少完美無缺的工作，例如程式碼生成、創意寫作、設計疊代或複雜的資料分析。透過提供結構化意見回饋的專門審查員代理程式，生成代理程式可以不斷修正工作，直到達到預先定義的完成條件為止，因此最終結果的品質和可靠性明顯高於單次傳遞方法。

重要概念：

- **{key} 與 {key?} 比較：**標準 `{key}` 語法會從共用狀態 (透過 `outputKey` 儲存) 插入內容。這裡使用的 `{key?}` 語法 (即 `{feedback?}`) 是迴圈的重要安全機制。系統會嘗試插入 `feedback` 的值，但如果尚未設定索引鍵 (例如在第一次疊代期間，審查員執行前)，問號會防止發生錯誤，並改為插入空字串。
- **maxIterations() 安全防護網：**設定 `maxIterations(n)` 的 `LoopAgent.builder()` 是防止無限迴圈的重要安全防護網。在 `LoopAgent` 工作流程中，這個週期會持續進行，直到審查人員呼叫 `exit_loop` 為止。如果審查員無法偵測到工作完成，迴圈可能會無限期執行。`maxIterations(n)` 方法可確保迴圈在設定的循環次數後終止。

11. 恭喜！



您已成功建構並探索各種 AI 代理，從簡單的對話式代理到複雜的多代理系統，您已瞭解 Java 適用的 ADK 核心概念：使用指令定義代理、透過工具賦予代理能力，以及將代理自動調度管理為強大的工作流程。

後續步驟

- 探索官方 [Java 適用的 ADK GitHub 存放區](#)。
 - 如要進一步瞭解這個架構，請參閱[說明文件](#)。
 - 請參閱這篇[網誌系列文章](#)，瞭解各種代理工作流程，以及[各種可用工具](#)。
 - 深入瞭解其他內建工具和進階回呼。
 - 處理背景資訊、狀態和構件，提供更豐富多元的互動體驗。
 - 導入並套用外掛程式，插入代理程式的生命週期。
 - 不妨試著建構專屬代理，解決現實世界中的問題！
-